Structured Query Language (SQL)

Ioana Manolescu
Inria Saclay and Ecole Polytechnique



Structured Query Language (SQL)

- *Data Manipulation Language* (*DML*)
  - posing queries, operating on tuples
- *Data Definition Language* (*DDL*)
  - operating on tables/views
- In this lecture, we will see:
  - Language features for expressing search conditions and desired search results (by example)
  - Conceptual evaluation strategy = SQL semantics

FINDING ANSWERS IN ONE TABLE

Tuples satisfying a condition

- **Finding tuples**: which are the sailors named ‘dustin’?

Sailors relation

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.0
58	rusty	10	55.0
44	ginny	8	36.0

SQL is case-insensitive!
Capitals for readability only.

Desired output

sid	sname	rating	age
22	dustin	7	45.0

Relational algebra

$\sigma_{name='dustin'}(Sailors)$

Tuple relational calculus

$\{ P \mid \exists P \in Sailors, P.name='dustin' \}$

SQL

```
SELECT *
FROM SAILORS
WHERE name='dustin';
```

Tuples satisfying a condition

- Which are the sailors not named ‘dustin’?

Sailors relation

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.0
58	rusty	10	55.0
44	ginny	8	36.0

Desired output

sid	sname	rating	age
31	lubber	8	55.0
58	rusty	10	55.0
44	ginny	8	36.0

```
SELECT *
FROM SAILORS
WHERE NOT(name= ’dustin’);

SELECT *
FROM SAILORS
WHERE name <> ‘dustin’;
```

Tuples satisfying a condition

- **Finding tuples**: which are the sailors rating at least 8 and younger than 40?

Sailors relation

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.0
58	rusty	10	55.0
44	ginny	8	36.0

Desired output

sid	sname	rating	age
44	ginny	8	36.0

Relational algebra

$\sigma_{rating \geq 8 \wedge age<40}(Sailors)$

Tuple relational calculus

$\{ P \mid \exists P \in Sailors \wedge (P.rating \geq 8) \wedge (P.age < 40) \}$

SQL

```
SELECT *
FROM SAILORS
WHERE rating >= 8 AND age < 40;
```

Conditions on unknown values

- Which are the sailors whose rating is unknown?
- Special constant in the SQL standard: NULL, typically used for «unknown» , or «not applicable»

Sailors relation

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	null	55.0
58	rusty	10	55.0
44	ginny	8	36.0

Desired output

sid	sname	rating	age
31	lubber	null	55.0

SQL

```
SELECT *
FROM SAILORS
WHERE rating IS NULL;
```

- We need the special keyword IS (and IS NOT) because conditions on null have special semantics (see next)

Conditions on unknown values

Sailors relation

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	null	55.0
58	rusty	10	55.0
44	ginny	8	36.0

Rationale: we don't know the rating of 31, thus we can't be sure it belongs to the result.

SELECT * FROM SAILORS WHERE rating < 8;

sid	sname	rating	age
22	dustin	7	45.0

SELECT * FROM SAILORS WHERE rating >= 8;

sid	sname	rating	age
58	rusty	10	55.0
44	ginny	8	36.0

More generally:

Comparator(op1, op2)	op1 is null	op1 is not null
op2 is null	false	false
op2 is not null	false	Depending on op1, op2, comparator

Conditions on unknown values

- Similarly:

Arithmetic(op1, op2)	op1 is null	op1 is not null
op2 is null	null	null
op2 is not null	null	Depending on op1, op2, arithmetic

SELECT max(rating, 9)
FROM SAILORS;

Sailors relation

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	null	55.0
58	rusty	10	55.0
44	ginny	8	36.0

max
9
null
10
9

Conditions based on belonging to a set

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	9	55.0
58	rusty	10	55.0
44	ginny	8	36.0

SQL

```
SELECT age, rating
FROM SAILORS
WHERE rating IN (10, 7);
```

Also: NOT IN

sid	sname	rating	age
22	dustin	7	45.0
58	rusty	10	55.0

Returning only some attributes

- What are the sailors' ages and ratings?

Sailors relation

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.0
58	rusty	10	55.0
44	ginny	8	36.0

Desired output

age	rating
45.0	7
55.0	8
55.0	10
36.0	8

Relational algebra

$\pi_{age,rating}$(Sailors)

Tuple relational calculus

{ R | ∃ P ∈ Sailors, R.rating=P.rating ∧ R.name=P.name }

SQL

```
SELECT age, rating
FROM SAILORS;
```

Returning only some attributes

- What are the sailors' ages ?

Sailors relation

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.0
58	rusty	10	55.0
44	ginny	8	36.0

Desired output

age
45.0
55.0
55.0
36.0

SQL

```
SELECT age
FROM SAILORS;
```

Multiset (or bag), not set of tuples!

To remove duplicates, use DISTINCT:

```
SELECT DISTINCT(age)
FROM SAILORS;
```

age
45.0
55.0
36.0

Combining selections and projections

- What are the age and rating of the sailors named ‘dustin’?

Sailors relation

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.0
58	rusty	10	55.0
44	ginny	8	36.0

Desired output

age	rating
45.0	7

Relational algebra

$$\pi_{age,rating}(\sigma_{\{name='dustin'\}}(\text{Sailors}))$$

Tuple relational calculus

{ R | ∃ P ϵ Sailors, R.rating=P.rating ∧ R.name=P.name }

SQL

```
SELECT *
FROM SAILORS
WHERE name='dustin';
```

Note: the SELECT clause defines the attribute order in the result

Controlling the output schema

- Return the sids as sailorid and age as sailorage

Sailors relation

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.0
58	rusty	10	55.0
44	ginny	8	36.0

Desired output

sailor-id	sailor-age
22	45.0
31	55.0
58	55.0
44	36.0

Relational algebra $\rho_{\{id \to sailorid, age \to sailorage\}}(\pi_{\{sid,age\}}(\text{Sailor}))$

TRC $\{P \mid \exists S \in Sailors \wedge P.sailorid = S.sid \wedge P.sailorage = S.age\}$

```
SELECT sid AS sailorid, age AS sailorage
FROM SAILORS;
```

Returning computation results

- For each sailor under 50, return the id, and rating plus 2.

Sailors relation

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.0
58	rusty	10	55.0
44	ginny	8	36.0

Desired output

sid	ratplus2
22	9
44	10

```
SELECT sid, (rating + 2) as ratplus2
FROM SAILORS
WHERE age<50;
```

```
SELECT SUBSTRING(sname, 1,3)
FROM SAILORS
WHERE age<50;
```

substring
dus
lub
rus
gin

https://www.postgresql.org/docs/current/functions.html

Semantics of a simple one-table query

SELECT DISTINCT attributes
FROM TABLE
WHERE condition

$$\pi_{attributes}(\sigma_{conditions}(Table))$$

SELECT DISTINCT (renamed attributes)

$$\rho_{att1\rightarrow n1,att2\rightarrow n2,\ldots}(\pi_{attributes}(\sigma_{conditions}(Table)))$$

A query without DISTINCT does not have a proper equivalent in rel. algebra/calculus unless it can be inferred that the result is duplicate-free

- E.g., if there is a clear PK for the result

Semantics of a simple one-table query

SELECT DISTINCT attributes
FROM TABLE
WHERE condition

$$\pi_{attributes}(\sigma_{conditions}(Table))$$

SELECT DISTINCT (renamed attributes)

$$\rho_{att1 \to n1, att2 \to n2, \ldots}(\pi_{attributes}(\sigma_{conditions}(Table)))$$

1. All the tuples of the **table**…
2. Filtered (σ) to satisfy the possible **conditions**…
3. From which we keep (π) only the **returned attributes**…
4. Possibly **renamed** (ρ) and/or we **compute** something from them

Result order in SQL

- Return the name and age of sailors rated level 8 or above *in increasing (decreasing) order of age*.

```
SELECT  sname, age
FROM  Sailors
WHERE  rating > 8
ORDER BY age DESC;

SELECT  *
FROM  Sailors
ORDER BY (rating, name) ASC;
```

In the relational algebra and calculus, we only work with *sets* of tuples → no notion of order

TOP-K Queries

- Return the name and age of the *ten youngest* sailors rated level 8 or above.

```
SELECT  sname, age
FROM  Sailors
WHERE  rating >= 8
ORDER BY age ASC
LIMIT 10;
```

AGGREGATING INFORMATION FROM A TABLE

Aggregating information in SQL

- So far we have returned
  - *(Some) individual tuples*
  - Attributes of *(some) individual tuples*
  - Renamed attributes of *(some) individual tuples*
  - Expressions over attributes of *(some) individual tuples*

- We can also compute aggregations over a whole table using aggregate functions: count, min, max, avg

Aggregation queries in SQL

```
SELECT  COUNT(*)
FROM  Sailors;
```

4

```
SELECT  COUNT(DISTINCT rating)
FROM  Sailors';
```

3

```
SELECT  AVG (age)
FROM  Sailors;
```

47.75

```
SELECT  COUNT(DISTINCT rating)
FROM  Sailors
WHERE age=55;
```

2

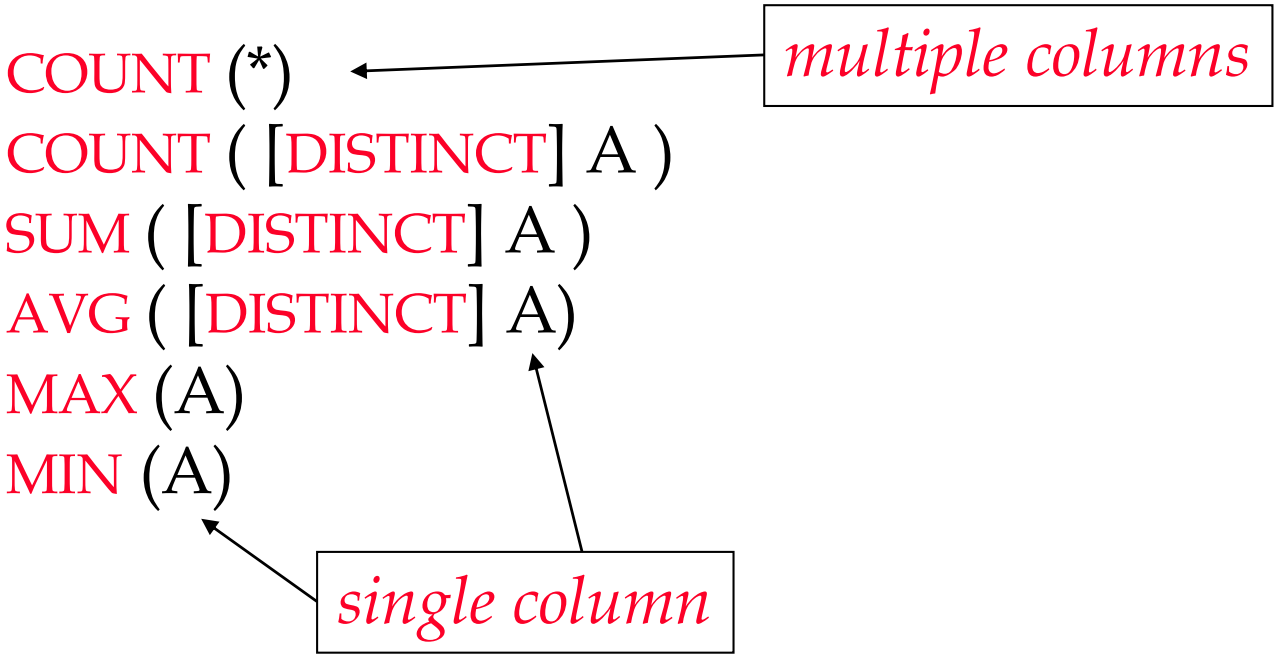
```
SELECT  AVG(DISTINCT age)
FROM  Sailors;
```

45.333

Each of these queries returns a single number (aggregated result), not a table

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.0
58	rusty	10	55.0
44	ginny	8	36.0

Aggregation functions



- Take a relation (single column or multiple columns), return a value.
- Significant extension of relational algebra.

SET OPERATIONS (UNION, DIFFERENCE, INTERSECTION)

Sample union

Sailors1

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

Sailors2

sid	sname	rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

```
SELECT  * FROM SAILORS1
UNION
SELECT * FROM SAILORS2;
```

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0
44	guppy	5	35.0
28	yuppy	9	35.0

UNION eliminates duplicates.

Sample union

Sailors1

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

Sailors2

sid	sname	rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

```
SELECT  * FROM SAILORS1
UNION ALL
SELECT * FROM SAILORS2;
```

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

UNION ALL keeps duplicates.

Sample set operations

Sailors1

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

Sailors2

sid	sname	rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

```
SELECT  * FROM SAILORS1
EXCEPT
SELECT * FROM SAILORS2;
```

sid	sname	rating	age
22	dustin	7	45.0

```
SELECT  * FROM SAILORS1
INTERSECT
SELECT * FROM SAILORS2;
```

sid	sname	rating	age
31	lubber	8	55.5
58	rusty	10	35.0

COMBINING MULTIPLE TABLES: CARTESIAN PRODUCT AND JOINS

Example instances and cartesian product

Sailors

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

Reserves

sid	bid	day
22	101	10/10/96
58	103	11/12/96

Cartesian product:

SELECT *
FROM Sailors, Reserves;

(sid)	sname	rating	age	(sid)	bid	day
22	dustin	7	45.0	22	101	10/10/96
22	dustin	7	45.0	58	103	11/12/96
31	lubber	8	55.5	22	101	10/10/96
31	lubber	8	55.5	58	103	11/12/96
58	rusty	10	35.0	22	101	10/10/96
58	rusty	10	35.0	58	103	11/12/96

Sample join (selection over cartesian product)

Range variables
Exactly TRC variables!

```
SELECT  S.sname
FROM    Sailors S, Reserves R
WHERE   S.sid=R.sid AND R.bid=103;
```

Result:

sname
rusty

(sid)	sname	rating	age	(sid)	bid	day		
22	dustin	7	45.0	22	101	10/10/96		✕
22	dustin	7	45.0	58	103	11/12/96	✕	
31	lubber	8	55.5	22	101	10/10/96	✕	✕
31	lubber	8	55.5	58	103	11/12/96	✕	
58	rusty	10	35.0	22	101	10/10/96	✕	✕
58	rusty	10	35.0	58	103	11/12/96		

Important: semantics of SQL queries including joins

Queries over one table

$\rho_{att1 \to n1,\ldots}(\pi_{attrs}(\sigma_{conds}(T)))$

1. All the tuples of the **table**…
2. Filtered (σ) to satisfy the possible **conditions**…
3. From which we keep (π) only the **returned attributes**…
4. Possibly **renamed** (ρ) and/or we **compute** something from them

Queries joining several tables

$\rho_{att1 \to n1,\ldots}(\pi_{attrs}(\sigma_{conds}(T_1 \times T_2 \ldots)))$

1. All the tuples of the **cartesian product of the tables**…
2. Filtered (σ) to satisfy the possible **conditions**…
3. From which we keep (π) only the **returned attributes**…
4. Possibly **renamed** (ρ) and/or we **compute** something from them

After (conceptually) building the cartesian product of all tables in the FROM clause, proceed exactly as for a 1-table query.

Three ways of expressing a join query

- SQL:
```
SELECT S.sname
FROM Sailors S, Reserves R
WHERE S.sid=R.sid AND R.bid=103;
```

- Relational algebra:

$$\pi_{sname}(Sailors \bowtie_{sid} \sigma_{bid=103}(Reserves))$$

- Tuple relational calculus:

$$\{ P \mid \exists S \in Sailors \wedge \exists R \in Reserves$$
$$(S.\text{sid}=R.\text{Rid} \wedge R.\text{bid}=103 \wedge P.\text{name}=S.\text{sname}) \}$$

Joins can be used to find tuples in a table…

… that have a certain relation with tuples from other tables.

E.g., find the IDs of sailors that have reserved at least a boat:

```
SELECT  DISTINCT(S.sid)
FROM  Sailors S, Reserves R
WHERE  S.sid=R.sid;
```

E.g., find the IDs of sailors that have reserved a red boat:

```
SELECT  DISTINCT(S.sid)
FROM   Sailors S, Reserves R, Boat B
WHERE  S.sid=R.sid and R.bid=B.bid and B.color='red'
```

Do not forget WHERE clause predicates

```
SELECT  S.name, B.color
FROM    Sailors S, Reserves R, Boats B
WHERE   S.sid=R.sid
```

- Algebraic expression for this query (ignoring duplicates):

$$\pi_{name,color}\left((Reserves \bowtie Sailors) \times Boats\right)$$

- No connection between Boats and the other tables
  - This is the typical query which runs "forever" ("query never seems to return")
    - In fact, DBMS quietly builds a huge Cartesian product
  - Cartesian product queries are almost never meant

More general/flexible joins: outerjoins

- Find all sailors and the dates of their reservations, if any

SELECT sid, sname, day

FROM Sailors LEFT OUTER JOIN Reserves ON Sailors.sid=Reserves.sid;

Algebra operator: $\exists\bowtie$

Sailors

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

Reserves

sid	bid	day
22	101	10/10/96
58	103	11/12/96

$$R \exists\bowtie_a S = (R \bowtie_a S) \cup \{(r, null_S) | r \in R \wedge \nexists s \in S \; s.t. (r.a = s.a)\}$$

Result

sid	sname	day
22	dustin	10/10/96
31	lubber	NULL
58	rusty	11/12/96

Also: RIGHT OUTER JOIN; $\bowtie\exists$

FULL OUTER JOIN $\exists\bowtie\exists$

NESTED SQL QUERIES

Nested SQL queries

In an SQL query, we can use:

- In the place of a **table**: an SQL query returning a **table**
- In the place of a constant: an SQL query returning a constant
- In the place of a set: an SQL query returning a set

The innermost query is said *nested within* the main one.

Semantics: the nested query is executed *once for each tuple generated by the outer query* (« function call »). In practice, often better!

Some nested queries cannot be expressed without nesting. The others can be also written without nesting.

Nested query in the place of a constant

- Find the names of the oldest sailors

```
SELECT sname
FROM Sailors S
WHERE S.age=(SELECT MAX(age)
             FROM Sailors);
```

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.0
58	rusty	10	55.0
44	ginny	8	36.0

- In the WHERE clause, S.age can be compared only with a constant (not a table!)
- SELECT MAX(age) FROM Sailors returns a constant ✔
- This query shape is the only way to answer such questions

Nested query in the place of a constant

- Find the names of the oldest sailors
- This syntax is WRONG:

```
SELECT sname, MAX(age)
FROM Sailors S;
```

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.0
58	rusty	10	55.0
44	ginny	8	36.0

- Intuition:
  - SELECT sname… FROM Sailors S computes something *from every Sailors tuple*
  - SELECT MAX(age) computes something *from all the Sailors tuples together ((multi)set)*
  - Cannot build a result tuple at the same time for every tuple and for a table.

Nested query in the place of a constant

- For each sailor, return their id, name, and number of reservations they made

```
SELECT S.sid, S.sname, (SELECT COUNT(*)
                        FROM Reserves R
                        WHERE R.sid=S.Sid)
FROM Sailors S;
```

- In this example, the nested query depends on the *outer* query, via the range variable S. This is often the case.
- We say the nested query is *correlated* with the outer one.

Nested query in place of a set: IN

Find the names of the sailors having reserved boat 103

Only a set can appear after IN.
Table of 1 column treated as a set!

```
SELECT   S.sname
FROM     Sailors S
WHERE    S.sid IN (SELECT R.sid
                   FROM Reserves R
                   WHERE R.bid = 103);
```

If two Sailors with same name booked 103, the name appears twice. A better query would find the ids and names:

```
SELECT   S.id, S.sname
FROM     Sailors S
WHERE    S.sid IN (SELECT R.sid
                   FROM Reserves R
                   WHERE R.bid = 103);
```

Nested query in place of a set: IN

Find the ids and names of the sailors having reserved boat 103

```
SELECT   S.id, S.sname
FROM     Sailors S
WHERE    S.sid IN (SELECT R.sid
                   FROM Reserves R
                   WHERE R.bid = 103);
```

Only a set can appear after IN.
Table of 1 column treated as a set!

Is the following query equivalent?

```
SELECT   S.id, S.sname
FROM     Sailors S, Reserves R
WHERE    S.sid=R.sid and R.bid=103
```

Nested query in the place of a set: (NOT) IN

Find the ids and names of sailors that have not reserved any boat

```
SELECT   S.id, S.sname
FROM     Sailors S
WHERE    S.sid NOT IN (SELECT R.sid
                       FROM Reserves R);
```

…that have reserved a red boat but no green boat:

```
SELECT   S.sid, S.sname
FROM     Sailors S
WHERE    S.sid IN (SELECT R.sid FROM Reserves R, Boat B
                   WHERE B.color='red');
AND S.sid NOT IN (SELECT R.sid FROM Reserves R, Boat B
                   WHERE B.color='green');
```

We can reuse R, B because they are in disjoint syntactic scopes.
S is visible (shared) from all the blocks (defined in outer query).

More SQL keywords for nesting a query in the place of a set: ALL, ANY

Find the ids and names of sailors that have not reserved any boat

```
SELECT   S.id, S.sname
FROM     Sailors S
WHERE    S.sid <> ALL (SELECT R.sid
                       FROM Reserves R);
```

Find the ids and names of sailors that did not make all the boat reservations in Reserves

```
SELECT   S.id, S.sname
FROM     Sailors S
WHERE    S.sid <> ANY (SELECT R.sid
                       FROM Reserves R);
```

More SQL keywords for nesting a query in the place of a set: ALL, ANY

Find the ids and names of sailors that made all the reservations

```
SELECT   S.id, S.sname
FROM     Sailors S
WHERE    S.sid = ALL (SELECT R.sid
                      FROM Reserves R);
```

Find the ids and names of sailors that made at least one reservation

```
SELECT   S.id, S.sname
FROM     Sailors S
WHERE    S.sid = ANY (SELECT R.sid
                      FROM Reserves R);
```

More SQL keywords for nested queries: (NOT) EXISTS

Find the ids and names of sailors that have not reserved any boat

The content of a SELECT clause within a NOT EXISTS does not matter much

```
SELECT   S.id, S.sname
FROM     Sailors S
WHERE    NOT EXISTS (SELECT *
                     FROM Reserves R
                     WHERE R.bid=S.bid);
```

As this example shows, ANY, ALL, (NOT) EXISTS generalize also to sets of tuples (subqueries→tables) of > 1 attribute.

Testing belonging, emptiness, or comparing to them (ANY/ALL) leads to boolean results which can be used in outer WHERE.

Another nested query with (NOT) EXISTS

- Find the sailors that have reserved two boats such that no other sailor has reserved the same two boats

```
SELECT   S.id, S.sname
FROM     Sailors S, Boats B1, Boats B2
WHERE    B1.bid<>B2.bid AND
         EXISTS (SELECT * FROM Reserves R1, Reserves R2
                 WHERE R1.sid=S.sid AND R2.sid=S.sid
                 AND R1.bid=B1.bid AND R2.bid=B2.bid)
         AND NOT EXISTS(
               (SELECT *
                FROM Reserves R1, Reserves R2, Sailors S2
                WHERE R1.sid=S2.sid AND R2.sid=S2.sid AND
                AND R1.bid=B1.bid AND R2.bid=B2.bid AND
                S2.sid<>S.sid);
```

Nested query in the place of a table: AS

- Nested query in the FROM clause:

```
SELECT G.sid, G.bid
FROM (SELECT S.age, R.*
        FROM Sailors S, Reserves R
        WHERE S.sid=R.sid and S.sname='ginny') AS G
WHERE G.age>40.0;
```

Nesting queries using Common Table Expressions (WITH… AS)

We can give a name to a query and treat it subsequently as a table (called **Common Table Expressions** or **CTEs**):

```
WITH top3Ages AS
        (SELECT DISTINCT(S.age) FROM Sailors S
         ORDER BY S.age DESCENDING LIMIT 3)
SELECT S.sid, S.age
FROM Sailors S
WHERE S.age in (SELECT age FROM top3Ages);
```

Union vs. OR

- Find sids of sailors having reserved a red or a green boat

```
SELECT DISTINCT R.sid
FROM Reserves R, Boats B
WHERE  R.bid=B.bid
AND (B.color= 'red' OR B.color= 'green' );
```

```
SELECT R.sid
FROM Reserves R, Boats B
WHERE  R.bid=B.bid
            AND B.color='red'
UNION
SELECT R.sid
FROM Reserves R, Boats B
WHERE  R.bid=B.bid
            AND B.color= 'green';
```

Intersect vs. AND

- Find sids of sailors having reserved a red and a green boat

```
SELECT DISTINCT  R1.sid
FROM   Reserves R1, Boats B1,
       Reserves R2, Boats B2
WHERE  R1.bid=B1.bid AND R2.bid=B2.bid
   AND (B1.color='red' AND B2.color='green')
   AND  R1.sid=R2.sid;
```

```
SELECT R.sid
FROM Reserves R, Boats B
WHERE  R.bid=B.bid
          AND B.color='red'
INTERSECT
SELECT R.sid
FROM Reserves R, Boats B
WHERE  R.bid=B.bid
          AND B.color='green';
```

Universal quantification

- Find the IDs and names of sailors who have reserved all the boats

```
SELECT  S.sid, S.sname
FROM    Sailors S
WHERE  NOT EXISTS
          ((SELECT  B.bid
            FROM  Boats B)
            EXCEPT
            (SELECT  R.bid
             FROM  Reserves R
             WHERE  R.sid=S.sid));
```

```
SELECT  S.sid, S.sname
FROM    Sailors S
WHERE  NOT EXISTS  (SELECT  B.bid
                    FROM  Boats B
                    WHERE  NOT EXISTS  (
                                SELECT  R.bid
                                FROM  Reserves R
                                WHERE  R.bid=B.bid
                                    AND R.sid=S.sid));
```

GROUPING

Motivation for grouping

- To apply aggregate operators to each *group* (subset) of tuples sharing some value(s)

- *Find the age of the youngest sailor for each rating level that at least two sailors share:*

```
SELECT  S.rating,  MIN (S.age) AS minage
FROM  Sailors S
GROUP BY  S.rating
HAVING  COUNT (*) > 1;
```

Result of a grouping query

```
SELECT  S.rating,  MIN (S.age)
                   AS minage
FROM  Sailors S
WHERE  S.age >= 18
GROUP BY  S.rating
HAVING  COUNT (*) > 1;
```

Result:

rating	minage
3	25.5
7	35.0
8	25.5

Sailors

sid	sname	rating	age
22	dustin	7	45.0
29	brutus	1	33.0
31	lubber	8	55.5
32	andy	8	25.5
58	rusty	10	35.0
64	horatio	7	35.0
~~71~~	~~zorba~~	~~10~~	~~16.0~~
74	horatio	9	35.0
85	art	3	25.5
95	bob	3	63.5
96	frodo	3	25.5

Queries with GROUP BY and HAVING

```
SELECT      [DISTINCT] target-list
FROM        relation-list
WHERE       qualification
GROUP BY    grouping-list
[HAVING     group-qualification];
```

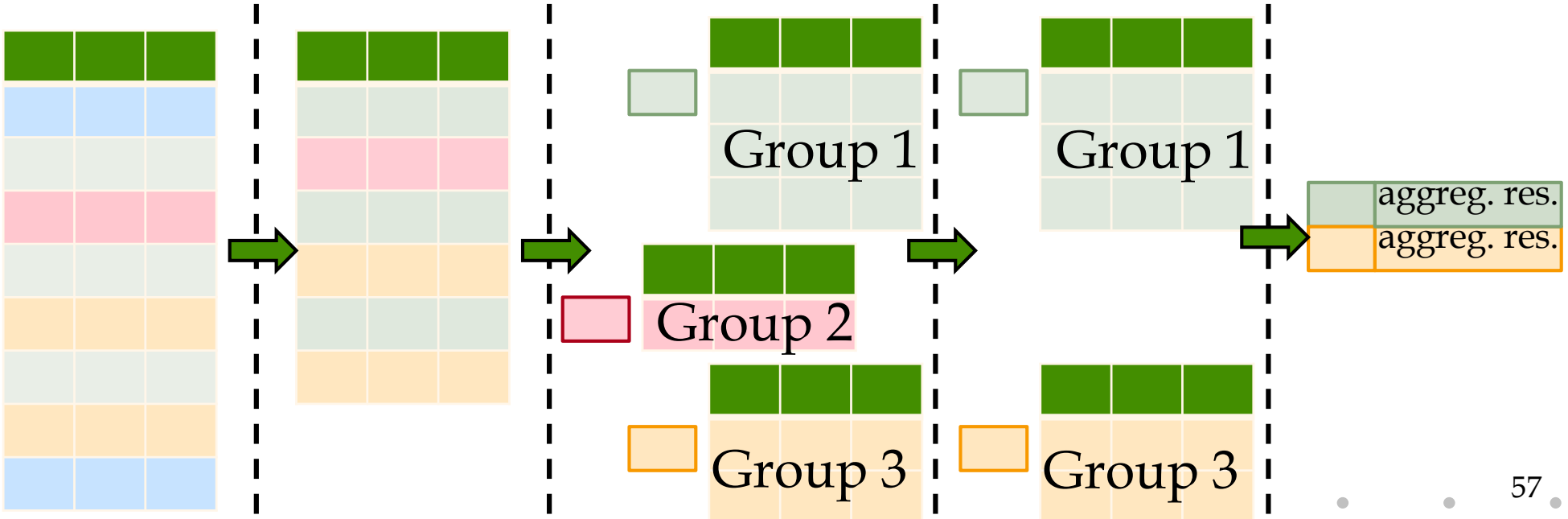
- A *group* is a set of tuples that have the same value for all attributes in *grouping-list*.
- Query returns a *single* answer tuple for each group.
- The *target-list* can only contain:
  (i) attributes in the *grouping-list* (e.g., *S.rating*), or
  (ii) *aggregate operations* on other attributes, e.g., MIN (*S.age*).

Semantics of a grouping query

- The cross-product of *relation-list* (FROM) is computed.
- Tuples that fail *qualification* (WHERE) are discarded.

} = join

- The remaining tuples are **partitioned** into **groups** by the value of attributes in *grouping-list* (GROUP BY)
- The *group-qualification* (HAVING) if present, **eliminates some groups**.
- A **single** answer tuple is produced (SELECT) for each qualifying group.



Extended relational algebra operator for grouping and aggregation: Γ

SELECT count(*)
FROM Sailors S
GROUP BY S.rating

$\Gamma_{\text{rating; } count}(\text{Sailors})$

SELECT S.rating, AVG (S.age)
FROM Sailors S
GROUP BY S.rating

$\Gamma_{\text{rating; } avg; \text{ age}}(\text{Sailors})$

SELECT count(*)
FROM Sailors

$\Gamma_{count}(\text{Sailors})$

More on group qualifications

- May also use HAVING SOME (condition)
  HAVING ANY(condition)
  - Interpreted within the group!

```
SELECT  S.rating,  MIN (S.age)
                          AS minage
FROM  Sailors S
WHERE  S.age >= 18
GROUP BY  S.rating
HAVING  SOME (S.name <> 'rusty');
```

Answer relation:

rating	minage
1	33.0
3	25.5
7	35.0
8	25.5
9	35.0

sid	sname	rating	age
22	dustin	7	45.0
29	brutus	1	33.0
31	lubber	8	55.5
32	andy	8	25.5
58	rusty	10	35.0
64	horatio	7	35.0
~~71~~	~~zorba~~	~~10~~	~~16.0~~
74	horatio	9	35.0
85	art	3	25.5
95	bob	3	63.5
96	frodo	3	25.5

Grouping combined with nesting

- Find those ratings for which the average age is the minimum over all ratings

```
SELECT  Temp.rating, Temp.avgage
FROM  (SELECT  S.rating, AVG (S.age) AS avgage
        FROM  Sailors S
        GROUP BY  S.rating) AS Temp
WHERE  Temp.avgage = (SELECT  MIN (Temp.avgage)
                      FROM  Temp);
```

UPDATING THE DATABASE

Creating tables

```
CREATE TABLE Sailors
(   sid INTEGER,
    sname CHAR(50) NOT NULL,
    rating INTEGER,
    age REAL,
    PRIMARY KEY  (sid));
```

```
CREATE TABLE Boats
(   bid INTEGER,
    bname CHAR (20),
    color CHAR(20),
    PRIMARY KEY  (bid)
    UNIQUE  (bname));
```

```
CREATE TABLE Reserves
(  sid INTEGER,
   bid INTEGER,
   day DATE,
    PRIMARY KEY  (sid,bid,day),
    FOREIGN KEY (sid) REFERENCES Sailors
            ON DELETE NO ACTION ON UPDATE CASCADE
    FOREIGN KEY (bid) REFERENCES Boats
            ON DELETE SET DEFAULT ON UPDATE CASCADE);
```

Destroying and altering tables

```
DROP TABLE Sailors;
```

- Destroys the Sailors relation, including schema and data.

```
ALTER TABLE Sailors
        ADD COLUMN credit_card:CHAR(40);
```

- The schema is altered by adding a new field; every tuple in the current instance is extended with a *null* value in the new field.

Updating a table

Three kinds of updates (modifications)

- Insert - create new tuple(s)
- Delete - remove existing tuple(s)
- Update - modify existing tuple(s)

Insertions

General form:

```
INSERT INTO R(A1,…., An)
       VALUES (v1,…., vn);
```

Example: Insert a new sailor to the database:

```
INSERT INTO Sailors(sid, sname, rating, age)
       VALUES (3212, 'Fred', 9, 44);
```

Can omit attributes; a missing attribute is set to *null*.
May omit attribute names if give values of all attributes in order.

Insertions

Example: Insert *multiple* tuples to Sailors:

```
INSERT   INTO   Sailors(sid, sname)

    SELECT  B.id,  B.name
    FROM      Boaters B
    WHERE   Boaters.rank = 'captain';
```

The query replaces the VALUES keyword.

Deletions

Example: delete all tuples that satisfy a condition

```
DELETE
FROM   Sailors
WHERE S.sname = 'Harry';
```

Using SQL, there is no way to delete only a single occurrence of a tuple that appears more than once in a relation.

Updates

Examples:

```
UPDATE  Employees
SET     salary = salary * 1.1;
```

```
UPDATE  Sailors S
SET     S.rating = s.rating + 1
WHERE   S.sid  IN
          (SELECT sid
           FROM   Reserves R
           WHERE  R.date ='Oct, 25');
```

Views

- A *view* is like a relation, but we store a *definition*, rather than a set of tuples.

```
CREATE VIEW RedBoatLovers (sid, name, bid)
    AS SELECT S.sid, S.sname, B.bid
       FROM   Sailors S, Reserves R, Boats B
       WHERE  S.sid = R.sid and R.bid = B.bid
              and  B.color='red' ;
```

- Views can be dropped using DROP VIEW command.
  - DROP TABLE if there's a view on the table? Use CASCADE to also drop the view, or RESTRICT to prevent removing the table as long as there is a view on it.

Uses of views

- Views can be used to present necessary information (or a summary), while *hiding details in underlying relation(s)*.
- **Logical data independence**
  - User application defined on a view is unchanged when underlying table changes
- **Computational benefits**
  - Result of a complex query is frequently used: CREATE MATERIALIZED VIEW RedBoatLovers…
  - The view content is stored just like a table
  - But: when the underlying tables change, the materialized view is automatically updated!
  - This may slow updates to those tables

CONSTRAINTS

Integrity Constraints (Review)

- Types of *integrity constraints* in SQL:
  - *Attribute constraints*: domain, NOT NULL
  - *Key constraints*: PRIMARY KEY, UNIQUE
  - *Foreign key constraints*: FOREIGN KEY
  - *General constraints*: CHECK, ASSERTION
- Inserts/deletes/updates that violate ICs are disallowed.

General Constraints

- Two forms: CHECK (single table constraint) and ASSERTION (multiple-table constraint).

```
CREATE TABLE   Sailors
        (   sid  INTEGER,
            sname  CHAR(50),
            rating  INTEGER,
            age  REAL,
            PRIMARY KEY  (sid),
            CHECK  ( rating >= 1
                        AND rating <= 10));
```

General Constraints (contd.)

- Can use queries to express constraints. Can also name constraints.

```
CREATE TABLE  Reserves
        (   sname  CHAR(10),
            bid  INTEGER,
            day  DATE,
            PRIMARY KEY  (bid,day),
            FOREIGN KEY (bid) REFERENCES Boats
                    ON DELETE CASCADE,
            CONSTRAINT  noInterlakeRes
            CHECK  ('Interlake' <>
                        ( SELECT  B.bname
                          FROM  Boats B
                          WHERE  B.bid=bid)));
```

Caution: CHECK over a different table not supported in some systems, e.g. Postgres

Constraints Over Multiple Relations

- Wrong! Table constraints attached to Sailors may never be enforced if Sailor remains empty (while Boats grows)

```
CREATE TABLE   Sailors
    ( sid  INTEGER,
      sname  CHAR(10),
      rating  INTEGER,
      age  REAL,
      PRIMARY KEY  (sid),
      CHECK
      ( (SELECT COUNT (S.sid) FROM Sailors S) +
        (SELECT COUNT (B.bid) FROM Boats B) < 100 );
```

Number of boats plus number of sailors is < 100

- ASSERTION is the right solution; not associated with either table.
- Assertion not supported in some systems

```
CREATE ASSERTION  smallClub
CHECK
( (SELECT COUNT (S.sid) FROM Sailors S) +
 (SELECT COUNT (B.bid) FROM Boats B) < 100 );
```

Triggers

- A trigger is a procedure automatically invoked by the DBMS in response to specific changes in the database.
  - Event: insert, delete, update
  - Condition: typically a query
    - boolean query
    - other queries: non empty interpreted as true
  - Action: execute new queries or updates, create new tables, change authorizations, ...
    - Has access to the condition query
    - Can refer to the old and new data

Trigger example

```
CREATE TRIGGER set_count AFTER INSERT ON STUDENTS
REFERENCING NEW TABLE AS InsertedTuples
FOR EACH STATEMENT
   INSERT INTO StatisticsTable(ModifTab, ModifType, Count)
              SELECT 'Students', 'Insert', COUNT *
              FROM InsertedTuples I
              WHERE I.age < 18
```

Event

Action

Also available: FOR EACH TUPLE

Syntax variations among vendors

Triggers activation

- When several triggers are activated by the same update, they will all be executed
  - The relative activation order is arbitrary!
- A trigger activation may activate another trigger!

- Setting triggers in an RDBMS is typically an advanced task, performed by the DBA

Triggers versus constraints

- Constraints and triggers can be used to preserve some properties of the databases (application logic)
- Constraints are more *declarative*
- Triggers are more *powerful*
  - E.g. insertions in Orders(itemID, qty, custID, price): a trigger may be used to fill in the price from the catalog
  - Further: allow a purchase even if price surpasses credit, if the client has a good payment history, and add the client to a table of candidates for credit increase.
- For this course, the objective is that you know how to use constraints to enforce application integrity.

Questions

